Optimization & Uncertainty in Learning Report
CS7641: Machine Learning
Spring 2026

1 Assignment Weight

The assignment is worth 12% of the total points.

Read everything below carefully as this assignment has changed term-over-term.

2 Objective

Most machine learning projects do not fail because a model is incapable of learning; they fail because we pull the wrong levers, in the wrong order, and then mistake coincidence for causation. This assignment asks you to stop treating your network as a black box. Our goal is to develop a clear, defensible understanding of how specific training choices drive specific outcomes. That means tracing cause to effect, not merely reporting a final accuracy.

Using the two **PyTorch neural networks you finalized in the SL Report as fixed backbones**, you will interrogate three forces that shape modern training:

1. **Randomized optimization (RO)** applied to the last one to three layers under a strict parameter budget to examine robustness, tail behavior, and local minima.
2. **Adam ingredient ablations** on the full network (Adult only) to separate the contributions of momentum, bias correction, adaptive scaling, and decoupled weight decay.
3. **Regularization** (standard Adam only, Adult only) to quantify how explicit regularization choices move the *test* metric, often by more than optimizer tweaks alone.

Every comparison is conducted under fixed, reported budgets (gradient evaluations, function evaluations, and wall-clock) so that differences are causal rather than accidental. You will conclude by composing a single, coherent training recipe from the elements that proved most effective on your data, and by explicitly accepting or rejecting your hypotheses with quantitative evidence. The intended outcome is a practitioner's decision rule you can carry into research or industry: when the model stumbles, you will know which knob to turn first, and why.

You will:

- Apply randomized optimization to the last 1–3 layers of your SL Report PyTorch neural networks under a strict parameter cap **on both datasets**.
- Dissect Adam via ablations on the full network **on Adult only** to measure speed-to-target, stability, and generalization.
- Run a targeted regularization study using standard Adam with your best Part 2 hyperparameters **on Adult only**.
- Integrate the winning pieces into a single best training recipe and explicitly accept or reject your hypotheses with quantitative evidence.

Hypothesis-driven analysis requirement

Your EDA should be consistent with what you reported in the SL Report so that comparisons remain meaningful. You may summarize EDA more briefly in this report, but you must preserve the same core facts and assumptions used to motivate your SL baseline. If you change anything material (cleaning rules, feature handling, label mapping, split strategy, or preprocessing), you must disclose and justify the change clearly and explain why it was necessary. Any such changes may skew comparisons to the SL baseline and must be acknowledged in your discussion.

After completing initial EDA for each dataset, you must propose **one clear hypothesis** about expected optimization behavior *before* running full experiments. These hypotheses must be grounded in theory from the lectures and readings (e.g., optimization stability, conditioning, momentum effects, adaptive step sizes, bias correction, regularization as implicit control, and robustness under stochastic training).

Your experiments and discussion should be structured around testing your hypotheses, not merely reporting results.

3 Required Datasets for Spring 2026

You will reuse the **same two datasets** from your SL Report. If these datasets are not used, you will receive a zero for the assignment.

Dataset A: Adult Income (Census Income)

- **Task type:** Binary classification
- **Target:** `income` ($\leq 50K$ vs $> 50K$)
- **Notes:** Expect class imbalance. Choose metrics that reflect this (F1 is required).

Dataset B: Wine Quality (Red and White)

- **Task type:** Multiclass classification
- **Target:** `quality` (discrete rating/class)
- **Notes:** Use Macro-F1 for multiclass evaluation and discuss per-class behavior.

4 Procedure (What you will do)

You are given two datasets and will complete **exactly two supervised learning tasks** total: **one per dataset**. For Spring 2026, both tasks are classification (Adult: binary; Wine: multiclass).

You must keep the **PyTorch SL Report backbone architecture fixed** as your baseline backbone. Only the regularization experiments in Part 3 may introduce or remove regularization modules, and any such changes must be explicitly documented.

General rules

- You may program in any language and are allowed to use standard libraries, **as long as they were not written specifically to solve this assignment**. Code repositories, notebooks, or AutoML templates created for CS7641 assignments ("plug-and-play" solutions) are **not allowed**.
- TAs must be able to recreate your experiments on a standard Linux machine if necessary.
- The analysis you provide in the report is paramount and the focus of the assignment.

PyTorch-only backbone requirement (important)

For this assignment, you must use **only the PyTorch neural networks** from your SL Report. This requirement exists to ensure ablation testing is controlled, comparable, and interpretable. Do not use the scikit-learn MLP from the SL Report for this assignment and there is not as much functionality for ablation testing.

5 Parts and Requirements

Part 1: Randomized Optimization on the last several layers (both datasets)

Goal: Evaluate whether search-based methods can improve the tail behavior of your PyTorch networks on **both datasets**.

- **Freeze:** Freeze all but the last 1–3 layers so the total trainable parameters for Randomized Optimization (RO) are $\leq\sim$50k (RO on $\gg$100k typically stalls). Report the exact trainable parameter count.
- **Objective:** Validation loss (include regularization terms if you are evaluating a "with regularization" condition).
- **Algorithms (all required):** Randomized Hill Climbing (RHC), Simulated Annealing (SA), Genetic Algorithm (GA).
- **Design disclosures:** Specify RHC restart policy and step-size schedule; SA initial temperature, decay factor, and step-size cooling; GA population size, selection, crossover, mutation rate, and whether elitism is used. Define the weight-perturbation distribution and scale and its adaptation schedule.
- **Accounting:** Count every objective call as a *function evaluation*. Adhere to the budgets defined by the assignment.

Part 2: Adam ablations on the full network (Adult only)

Scope constraint (required): All Part 2 Adam ablation experiments must be run on the **classification task** for this project. For Spring 2026, this is the **Adult Income** dataset.

Why Adult only: Part 2 is the most compute-heavy component. Restricting it to one dataset ensures you can run enough seeds and sweeps to make conclusions credible. Wine will be revisited later in the Unsupervised Learning unit.

Train the same backbone with the following optimizers (Adult only):

- **SGD (no momentum)**
- **SGD with Momentum**
- **Nesterov momentum**
- **Adam (baseline)**
- **Adam (no bias correction)**
- **Adam with** $\beta_1 = 0$ (RMSProp-like)
- **AdamW** (decoupled weight decay)

Required analyses (Adult only):

- **Time and steps to a fixed validation-loss threshold** ℓ (define once and use consistently).
- **Sensitivity heatmaps** over (α, β_1) and (α, β_2) on coarse grids.
- **Stability across many seeds** (variance bands or median and IQR).
- **Generalization gap** (train vs validation at budget).

Part 3: Regularization study (standard Adam only; Adult only)

Goal: measure how much regularization improves generalization under a fixed optimizer and fixed compute.

Scope restriction (required): Run Part 3 on **Adult Income only**.

Optimizer constraint (required): Use **standard Adam (baseline)** with the **best Part 2 hyperparameters** for Adult. Do not change Adam settings in this part.

Backbone constraint (required): Use the **PyTorch SL Report backbone**. Only add/remove regularization modules and document changes.

Required techniques (do all four):

1. **L2 weight decay (coupled):** select and justify a search strategy; report the chosen setting.
2. **Early stopping:** define and justify a stopping criterion (e.g., patience).
3. **Dropout:** insert at appropriate layers; document placement; select and justify dropout rates.
4. **Noise regularization:** evaluate **label smoothing** and **modest input noise/augmentation** (training-only) and justify magnitudes.

Presentation (Adult only):

- **Baseline:** Adam (baseline), no added regularization.
- **Best single regularizer (Adam):** best individual technique under the fixed budget.
- **Best combination (Adam):** best combination under the same fixed budget.

Keep compute budgets identical across comparisons and make explicit how much regularization moved the *test* metric relative to Part 2.

Part 4: Integrated Best Combination (Extra Credit)

Optional (up to 5 points). If you choose to complete Part 4, you must **integrate all three components** and report how the final recipe compares to the rest of your results.

What to do (all required):

- **Optimizer:** Use **standard Adam** with the **best hyperparameters from Part 2** (same batch size).
- **Regularization:** Apply the **best combination from Part 3**, with exact values and layer placements.
- **RO fine-tuning:** Apply your **best-performing RO algorithm from Part 1** to the **same last 1–3 layers** under the **same parameter cap** used in Part 1.

Constraints (to keep this a composition, not a new search):

- Do not introduce new optimizers or fresh wide hyperparameter sweeps.
- Limit RO fine-tuning to $\leq 10\%$ of the Part 1 RO function-evaluation budget and ≤ 5 small interaction trials (for example, dropout rate adjustments).
- Keep seeds, hardware class, data splits, and batch size consistent with earlier parts.

What to report explicitly: Whether RO fine-tuning on top of the best Adam and regularization **improves test metrics** beyond the best from Parts 1–3; the **compute trade-off** (test metric vs total evaluations and time); and any stability effects you observed (variance across seeds). End with a concise hypothesis resolution paragraph that accepts or rejects your hypotheses with quantitative evidence.

6 What is required as you go through your analysis

The following applies to both datasets for Part 1 (RO), and applies to Adult for Parts 2 and 3. Extra Credit composition rules apply only if attempted.

Data and targets

- Declare the target and justify your metrics.
- Use the **same fixed** train, validation, and test splits from the SL Report; document preprocessing and prevent leakage.

Methodology and splits

- One held-out test used once at the end; tune only on train and validation (cross-validation or hold-out).
- Set and log all random seeds; record exact splits and any deterministic flags.
- Keep the **PyTorch SL backbone architecture fixed**; only Part 3 may add or remove regularization modules (document precisely).

Compute accounting and fairness

- **Report** gradient evaluations (SGD and Adam-family), function evaluations (RO), and wall-clock on the same hardware class.
- Respect assignment budgets; clearly mark any over-budget runs and exclude them from head-to-head tables.
- Keep batch size, hardware class, and data splits consistent across methods unless a change is essential (justify and log).

Required figures and tables

- **Loss vs compute:** Validation loss vs wall-clock and vs evaluations (gradient or function).
- **RO progress (Part 1):** Best-so-far objective vs function evaluations, with operator settings in the caption (both datasets).
- **Sensitivity heatmaps (Part 2, Adult only):** (α, β_1) and (α, β_2) for Adam variants (coarse grids).
- **Stability bands (Part 2, Adult only):** Median and IQR over many seeds for optimizer trajectories.
- **Regularization sweep (Part 3, Adult only):** Test metric across individual regularizers and the best combination under identical budgets.
- **Summary table:** Method, Best Val Loss, Test Metric, Time to ℓ, #Grad Evals, #Func Evals, Updates.
- **If doing Extra Credit (Part 4):** Final comparison panel and a frontier plot (test metric vs total compute).

Analysis and conclusion

- Make claims only with budget-matched evidence (include dispersion, not just means). Explain failures (divergence, plateaus) and attribute causes.
- End with a **decision rule** for Adult: when to prioritize optimizer settings vs regularization vs RO; state limits of transfer.
- Explicitly compare outcomes back to the SL Report baseline test results, and state whether each intervention improved, did not improve, or improved only under certain stability and compute conditions.

7 External Sources and Literature (required)

- Include outside sources beyond course materials, with at least **two peer-reviewed references**.
- Use these sources to **justify choices** (metrics for imbalance, threshold selection, calibration, optimizer behavior, stability interpretation, leakage controls) and to **interpret results**.
- Acceptable citation styles: **MLA, APA, or IEEE**. Pick one and be consistent.

8 Report Requirements

Your report is a formal technical paper limited to **8 pages total**, including figures and references. Anything past 8 pages will not be read.

Your report must be written in LaTeX on Overleaf. When submitting your report, you are required to include a **READ-ONLY** link to the Overleaf Project. If a link is not provided in the report or Canvas submission comment, 5 points will be deducted from your score. Do not share the project directly via email.

Interpret, don't summarize

Every figure and table must include a takeaway tied to optimizer behavior, regularization behavior, RO dynamics, and the dataset. Avoid repeating the legend; explain what the result *means* and why it happened.

Formal writing requirement

Your *report* must read like a technical paper, not a checklist. Bullets are allowed when they improve clarity (e.g., listing hyperparameters, summarizing preprocessing steps, or describing an experimental protocol), but your **analysis and conclusions must be written in paragraphs** with coherent reasoning. Excessive bullet-only writing that replaces interpretation (especially in Results/Discussion) will be treated as a draft and scored accordingly. As a rule of thumb: if a section could be pasted into a spreadsheet and still make sense, it is not sufficient prose. If there are any use of excess bullets, we will deduct 50 points from the report total.

9 Acceptable Libraries

You may use any standard open-source libraries that support reproducible experimentation and were not written specifically to solve this assignment. The list below includes common examples.

Recommended stack (examples)

- **Core ML:** `scikit-learn` (splits, metrics, calibration, utilities), `imbalanced-learn` (imbalance utilities).
- **PyTorch:** required for the neural networks and ablation work in this assignment.
- **pyperch:** helpful for parameter search and includes PyTorch-compatible optimizers modeled after ABAGAIL. It also provides an alternative to the mlrose neural network optimizers.
- **Data and plotting:** `pandas`, `NumPy`, and `matplotlib` (or `plotly`/`altair`/`seaborn`).

Randomized Optimization libraries (optional)

- **mlrose-hiive** (Python) — RHC, SA, GA.
- **DEAP** (Python) — evolutionary algorithms (GA tooling).
- **Nevergrad** (Python) — derivative-free optimization toolbox (use only for RHC, SA, GA-like operators you disclose).

If you use any library tooling, you must still disclose operator choices and count every objective call as a function evaluation.

10 Submission Details

All scored assignments are due by the time and date indicated on Canvas (Eastern Time, ET). Canvas displays times in your local time zone if your profile is set correctly; grading deadlines are enforced in ET.

All assignments are due at 11:59:00 PM ET on the final Sunday of the unit. Submissions received by **7:59:00 AM ET** the next morning are accepted **without penalty**.

Late penalty

After the grace window, the score is reduced **20 points per calendar day**. Treat **11:59 PM ET** as your real deadline; allow time for upload and verification.

What to submit

You will submit **two PDFs**:

1. **OL_Report_{GTusername}.pdf** — your report (Overleaf).
2. **REPRO_OL_{GTusername}.pdf** — your reproducibility sheet, which must include:
   (a) A **READ-ONLY** link to your Overleaf project.
   (b) A **GitHub commit hash** (single SHA) from the final push of your code.
   (c) **Exact run instructions** to reproduce results on a standard Linux machine (environment setup, commands, data paths, and random seeds).
   (d) **EDA summary confirmation** for both datasets (Adult and Wine).
   Note: EDA should match the SL Report unless changes are justified and explicitly disclosed.

- Include the READ-ONLY Overleaf link in the report or Canvas submission comment. **Do not send email invitations.**
- Use the **GT Enterprise GitHub** for course-related code.
- Provide sufficient instructions to retrieve code and data (Canvas paths and file names are sufficient).

11 Feedback Requests

When your assignment is scored, you will receive feedback explaining errors and successes. If you are confused by feedback, start a private thread on Ed.

12 Plagiarism and Proper Citation

The easiest way to fail this class is to plagiarize. **Using the analysis, code, or graphs of others in this class is considered plagiarism**. We care about your analysis: it must be original and grounded in your own experiments.

If you copy any amount of text from other students, websites, or any other source without proper attribution, that is plagiarism. Citing is required but does not permit copying large blocks of text. All citations must use a consistent style (IEEE, MLA, or APA).

LLMs (disclosure required)

We treat AI based assistance the same way we treat collaboration with people. You may discuss ideas and seek help from classmates, colleagues, and AI tools, but all submitted work must be your own. The goal of reports is synthesis of analysis, not merely getting an algorithm to run.

Every submission must include an AI Use Statement. List the tools used and what they assisted with and confirm that you reviewed and understood all assisted content.

Allowed with disclosure: brainstorming, outlining, grammar and clarity edits, code generation, code refactoring, and debugging.

Not allowed: submitting AI written analysis, conclusions, or figures as your own; fabricating results or citations; paraphrasing AI or prior work to evade checks.

Example Statement at the very end of the report before References: "AI Use Statement. I used ChatGPT and Visual Studio Code Copilot to brainstorm and outline sections of the report, generate and refactor small code snippets, debug an indexing issue, and edit grammar and clarity throughout. I reviewed, verified, and understand all assisted content."

13 Version Control

- v2.0 – 01/22/2026 – TJL added pyperch as a recommendation.
- v1.0 – 01/16/2026 – TJL finalized OL Report for Spring 2026 term.

Assignment description written by Theodore LaGrow.